

```

/usr/local/include
/usr/local/gupc/include
/usr/include
End of search list.

GNU UPC (GCC) version 4.8.0 20130311 (GNU UPC 4.8.0-3)
(x86_64-redhat-linux)
compiled by GNU C version 4.8.0 20130311 (GNU UPC 4.8.0-3),
GMP version 5.0.5, MPFR version 3.1.1, MPC version 0.9
GGC heuristics: --param ggc-min-expand=100 --param ggc-min-
heapsize=131072
Compiler executable checksum:
9db6d00c84dee663b5eb4965bf5012f
COLLECT_GCC_OPTIONS='-o' 'hello' '-v' '-fupc-link'
'-mtune=generic' '-march=x86-64'
as -v --64 -o /tmp/ccSYlmb.o /tmp/ccTdo4Ku.s
...
COLLECT_GCC_OPTIONS='-o' 'hello' '-v' '-fupc-link'
'-mtune=generic' '-march=x86-64'
...

```

The `-g` option will generate debug information. To output debugging symbol information in DWARF-2 (Debugging With Attributed Record Formats), use the `-dwarf-2-upc` option. This can be used with GDB-UPC, a GNU debugger that supports UPC.

The `-fupc-debug` option will also generate the filename and the line numbers in the output.

The optimisation levels are similar to the ones supported by GCC: `'-O0'`, `'-O1'`, `'-O2'`, and `'-O3'`.

Variables that are shared among threads are declared using the `'shared'` keyword.

Examples include:

```

shared int i;
shared int a[THREADS];
shared char *p;

```

`'THREADS'` is a reserved keyword that represents the number of threads that will get executed during run-time. Consider a simple vector addition example:

```

#include <upc_relaxed.h>
#include <stdio.h>
shared int a[THREADS];
shared int b[THREADS];
shared int vsum[THREADS];
int
main()
{
    int i;
    /* Initialization */
    for (i=0; i<THREADS; i++) {
        a[i] = i + 1;          /* a[] = {1, 2, 3, 4, 5}; */

```

```

        b[i] = THREADS - i;    /* b[] = {5, 4, 3, 2, 1}; */
    }
    /* Computation */
    for (i=0; i<THREADS; i++)
    if (MYTHREAD == i % THREADS)
    vsum[i] = a[i] + b[i];
    upc_barrier;
    /* Output */
    if (MYTHREAD == 0) {
        for (i=0; i<THREADS; i++)
            printf("%d ", vsum[i]);
    }
    return 0;
}

```

`'MYTHREAD'` indicates the thread that is currently running. `upc_barrier` is a blocking synchronisation primitive that ensures that all threads complete before proceeding further. Only one thread is required to print the output, and `THREAD 0` is used for this. The program can be compiled and executed using:

```

# gupc vector_addition.c -o vector_addition
# ./vector_addition -n 5
6 6 6 6 6

```

The computation loop in the above code can be simplified with the `upc_forall` statement:

```

#include <upc_relaxed.h>
#include <stdio.h>
shared int a[THREADS];
shared int b[THREADS];
shared int vsum[THREADS];
int
main()
{
    int i;
    /* Initialization */
    for (i=0; i<THREADS; i++) {
        a[i] = i + 1; /* a[] = {1, 2, 3, 4, 5}; */
        b[i] = THREADS - i; /* b[] = {5, 4, 3, 2, 1}; */
    }
    /* Computation */
    upc_forall (i=0; i<THREADS; i++; i)
    vsum[i] = a[i] + b[i];
    upc_barrier;
    if (MYTHREAD == 0) {
        for (i=0; i<THREADS; i++)
            printf("%d ", vsum[i]);
    }
    return 0;
}

```

The `upc_forall` construct is similar to a `for` loop, except